

Individual Project Report - UCAB Platform

Full-Stack Developer & Documentation Specialist Contribution

1. Project Metadata

Project Title: UCAB - Full-Stack Cab Booking & Ride-Hailing Platform

Author: Konatham Sreevanth

My Role: Full-stack Developer and Documentation Specialist

Team Collaborators: Niharika Devi Guttula, Madiki Devi, Mahipala Veera Venkata Manikanta, and Naveen Kumar Kondepudi

2. Executive Summary & Project Scope

The UCAB application is a web-based, full-stack cab booking and taxi dispatch software designed to streamline passenger-driver connectivity. The platform manages JWT authentication, user and driver dashboards, simulated payment processing, and customer support channels. As a Full-stack Developer and Documentation Specialist on the team, my primary objective was to coordinate full-stack feature development, implement test-verification pipelines, ensure API compatibility, and compile the technical layout of the system.

3. My Key Contributions & Work Accomplished

- **Geolocation & Geocoding Integrations:** Integrated live HTML5 Geolocation API parameters to resolve passenger locations; hooked up Nominatim search engine to query dynamic location addresses in English/Telugu.
- **Dynamic Street Curves Routing:** Configured OSRM street paths parsing logic into LeafletMap to draw routes tracing curves instead of straight lines, and set smooth linear transitions on vehicle markers.
- **Google Maps Wrapper Implementation:** Architected a unified mapping container supporting dynamic switches between open-source Leaflet and official Google Maps wrappers via environment variables.
- **Persistent JSON Backups Synchronizer:** Programmed local JSON backups file data sync handlers on backend authentication controllers, preventing registration resets upon DB restarts.
- **Error Diagnosis & Code Debugging:** Resolved script-reloading loops in useJsApiLoader hook, debounced autocomplete suggestions, fixed payment schema enums validation checks, and corrected CSS stacking navbar panels.

4. System Architecture Overview

Frontend Stack: Built with React.js (Vite framework) with responsive styled dashboards, centralized contexts for global auth state, Google Maps SDK wrapper, and Axios.

Backend Stack: Node.js and Express.js REST APIs utilizing custom middleware for JWT generation, route guarding, persistent JSON database logs sync handlers, and unified error handling.

Database Layer: MongoDB Atlas cloud storage using Mongoose ODM schemas for User, Driver, Ride, Payment, and Support models.

5. Local Environment Setup & Run Procedures

Setup Phase: Install Node.js (v18+) and execute 'npm run install-all' from the project root to configure dependencies in both directories. Add environment variables for cloud DB and maps keys.

Execution Phase: Run 'npm run dev' to concurrently launch the Express REST API and Vite server. Access the dashboards on local ports and test authentication flows.

6. Specific Verification Scenarios Completed

User Authentication: Validated password hashing on registration, JWT token generation upon login, and route protections for drivers and administrators.

Ride Booking Workflow: Created mock ride bookings from passenger dashboards, checked database insertions, verified availability changes, and updated trip statuses to completion.

Simulated Payment & Support: Tested fare collection triggers and simulated payment state updates; validated support ticketing logs for customer queries.

7. Future Goals & Roadmap

For future releases, my focus will be on integrating real-time coordinate streaming using WebSockets (Socket.io), adding production Stripe payment gateway integrations, and wrapping the responsive React dashboard within a cross-platform React Native app.